PRÁCTICA DE CIFRADO SIMÉTRICO EN UBUNTU 26

Obxectivo

Aprender a protexer a confidencialidade dun ficheiro utilizando un cifrado simétrico baseado nun contrasinal.

Comprender o ciclo de vida dun ficheiro cifrado: creación, comprobación, cifrado, retirada da copia en claro, descifrado e verificación final.

> Idea clave: o cifrado simétrico funciona como un cadeado cunha única chave. A mesma chave -neste caso, un contrasinal- úsase tanto para cifrar como para descifrar. Se perdemos ese contrasinal, podemos perder o acceso aos datos.

A ferramenta que usaremos é GnuPG (gpg), dispoñible en Ubuntu. Nesta práctica indicaremos expresamente o algoritmo AES-256 para que todo o alumnado traballe coa mesma configuración.

Para poñernos en contexto, pensa que es un axente de seguridade e acabas de recibir información vital sobre unha ameaza. Debes gardala nun ficheiro de texto e asegurarte de que ninguén poida lela sen coñecer o contrasinal.

A túa misión

1. Preparar o contorno: comprobar que gpg está instalado e crear unha carpeta de traballo.
2. Crear a Evidencia: crear un ficheiro chamado informe_ameaza.txt coa mensaxe indicada.
3. Comprobar a Evidencia: visualizar o ficheiro e gardar un resumo SHA-256 para poder verificar despois que o contido recuperado é idéntico.
4. Protexer a Evidencia: cifrar o ficheiro con AES-256 e un contrasinal elixido por ti. O resultado será informe_ameaza.txt.gpg.
5. Retirar a copia en claro: borrar o ficheiro orixinal para que non quede dispoñible na carpeta de traballo.
6. Recuperar a Evidencia: descifrar o ficheiro .gpg e gardalo como informe_recuperado.txt.
7. Verificar o resultado: comprobar que a mensaxe recuperada coincide coa orixinal e que o seu SHA-256 é o mesmo.

Solución Guiada (Pasos para o Alumno)

> Antes de empezar: abre unha terminal con Ctrl+Alt+T. Executa os comandos un por un e comproba o resultado de cada paso. Non copies todos os comandos á vez.

Paso 0: Comprobar que GnuPG está instalado

Escribe o seguinte comando:

```
gpg --version
```

Se aparece información sobre a versión de GnuPG, podes continuar. Se aparece unha mensaxe similar a command not found, instálao con:

```
sudo apt update
sudo apt install gnupg
```

> IMPORTANTE: durante a instalación Ubuntu pode pedirche o contrasinal do teu usuario. Ao escribilo na terminal non verás asteriscos nin caracteres; é normal.

Paso 1: Crear unha carpeta de traballo

Para non mesturar os ficheiros desta práctica con outros documentos, crea unha carpeta específica e entra nela:

```
mkdir -p ~/cifrado_simetrico
cd ~/cifrado_simetrico
```

Comprobación: comproba onde estás:

```
pwd
```

A ruta debería rematar en /cifrado_simetrico.

Paso 2: Crear o Ficheiro de Texto

Crea o ficheiro coa información secreta:

```
echo "A vulnerabilidade principal está no servidor web desactualizado." >
informe_ameaza.txt
```

Comprobación: visualiza o seu contido:

```
cat informe_ameaza.txt
```

Deberías ver exactamente a frase anterior. Tamén podes comprobar que o ficheiro existe con:

```
ls -l
```

Paso 3: Gardar unha pegada SHA-256 do ficheiro

Antes de cifrar, imos calcular unha pegada ou hash SHA-256. Non serve para ocultar o contido: úsase para comprobar máis tarde se o ficheiro recuperado é exactamente igual ao orixinal.

```
sha256sum informe_ameaza.txt
```

Verás unha cadea longa de números e letras seguida do nome do ficheiro. Garda só o hash nun ficheiro:

```
sha256sum informe_ameaza.txt | cut -d ' ' -f1 > hash_orixinal.txt
```

Comprobación: comproba o que se gardou:

```
cat hash_orixinal.txt
```

Paso 4: Cifrar o Ficheiro

Agora cifraremos o ficheiro usando AES-256 e un contrasinal. Executa:

```
gpg --symmetric --cipher-algo AES256 \
--output informe_ameaza.txt.gpg informe_ameaza.txt
```

> IMPORTANTE: Ubuntu pode mostrar a petición do contrasinal nunha xanela gráfica en lugar de facelo directamente na terminal. Isto depende do compoñente pinentry que use o sistema.

Introduce un contrasinal que poidas lembrar durante a práctica e repíteo cando cho solicite. Para unha práctica de aula pode ser sinxelo; nun sistema real debería ser longo e difícil de adiviñar.

Comprobación: executa:

```
ls -l
```

Agora deberían aparecer, entre outros, informe_ameaza.txt e informe_ameaza.txt.gpg.

Podes comprobar tamén que o ficheiro cifrado non é texto normal:

```
file informe_ameaza.txt.gpg
```

Se executas cat informe_ameaza.txt.gpg aparecerán datos binarios ou caracteres sen sentido. Isto só indica que non é texto plano; a protección real vén do cifrado e do contrasinal.

Paso 5: Retirar o Ficheiro Orixinal

Unha vez comprobado que existe o ficheiro cifrado, elimina da carpeta a copia en claro:

```
rm informe_ameaza.txt
```

Comprobación: comproba os ficheiros que quedan:

```
ls -l
```

Xa non debería aparecer informe_ameaza.txt, pero si informe_ameaza.txt.gpg e hash_orixinal.txt.

> ATENCIÓN SOBRE O BORRADO: o comando rm elimina a referencia ao ficheiro no sistema de ficheiros, pero non garante un borrado físico seguro dos datos. En discos SSD, máquinas virtuais e sistemas modernos, ferramentas como shred tampouco poden garantir que todas as copias internas desaparezan. Nesta práctica usamos rm para aprender a non deixar unha copia en claro accesible, non como demostración de borrado forense seguro.

Paso 6: Forzar unha nova petición de contrasinal (opcional)

GnuPG utiliza un axente que pode lembrar temporalmente o contrasinal. Se ao descifrar non cho volve pedir, non significa que o ficheiro non estea protexido: pode estar usando a caché de gpg-agent. Para limpala antes de continuar podes executar:

```
gpgconf --kill gpg-agent
```

Este paso é opcional, pero resulta útil na aula para comprobar que realmente coñeces o contrasinal.

Paso 7: Descifrar o Ficheiro

Recupera a información e gárdaa nun novo ficheiro chamado informe_recuperado.txt:

```
gpg --output informe_recuperado.txt --decrypt informe_ameaza.txt.gpg
```

Introduce o mesmo contrasinal que utilizaches ao cifrar. Se é correcto, GnuPG creará o novo ficheiro.

Comprobación: le o contido recuperado:

```
cat informe_recuperado.txt
```

Na pantalla debería aparecer: A vulnerabilidade principal está no servidor web desactualizado.

Paso 8: Verificar que o ficheiro recuperado é idéntico

Calcula a pegada SHA-256 do ficheiro recuperado:

```
sha256sum informe_recuperado.txt | cut -d ' ' -f1
```

Agora compáraa coa pegada que gardaches ao principio:

```
cat hash_orixinal.txt
```

As dúas cadeas deberían ser exactamente iguais. Se coinciden, temos unha comprobación forte de que o contido recuperado é o mesmo que o orixinal.

Tamén podes facer a comprobación automaticamente:

```
test "$(sha256sum informe_recuperado.txt | cut -d ' ' -f1)" = "$(cat
hash_orixinal.txt)" && \
echo "CORRECTO: os hashes coinciden" || echo "ERRO: os hashes non coinciden"
```

Paso 9: Probar un contrasinal incorrecto

Para comprobar o comportamento do sistema, podes volver eliminar temporalmente o ficheiro recuperado e limpar a caché:

```
rm informe_recuperado.txt
gpgconf --kill gpg-agent
```

Tenta descifrar de novo pero introduce un contrasinal incorrecto. GnuPG debería indicar que o descifrado fallou. Despois repite o comando usando o contrasinal correcto.

> NON BORRES: o ficheiro .gpg ata rematar a práctica. É a copia cifrada que permite recuperar a información.

Que aprendemos?

- Cifrado simétrico: a mesma clave ou contrasinal intervén no cifrado e no descifrado.
- AES-256: é o algoritmo que empregamos explicitamente nesta práctica.
- Ficheiro en claro: pode lerse directamente e debe protexerse se contén información sensible.
- Ficheiro cifrado: non pode recuperarse de forma útil sen a clave correcta.
- SHA-256: permite comprobar a integridade dun ficheiro; non é un método de cifrado.
- rm: elimina o ficheiro da vista normal do sistema, pero non equivale necesariamente a un borrado físico irrecuperable.

Cuestións finais

1. Por que este método se chama cifrado simétrico?
2. Que diferenza hai entre cifrar un ficheiro e calcular o seu SHA-256?
3. Que ocorrería se esqueceses o contrasinal?
4. Por que non debemos deixar unha copia do ficheiro en claro xunto á copia cifrada?
5. Por que o comando rm non pode considerarse sempre un borrado seguro dos datos?
6. Que vantaxe ten indicar explicitamente AES256 no comando desta práctica?